Introduction to Embedded Systems

Course Code: 244066

Chapter 5: STM32 Serial Communication Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Contents

5.1. Introduction to Serial Communication

- Basics of Serial Communication
- Asynchronous Serial Protocol

5.2. USART in STM32

- STM32 USART Overview
- USART Block Diagram
- USART Registers

5.3. Serial Communication Programming

- Initialization Steps for USART
- Sending Data (TX) Flow
- Receiving Data (RX) Flow

5.4. Summary

5.1. Introduction to Serial Communication

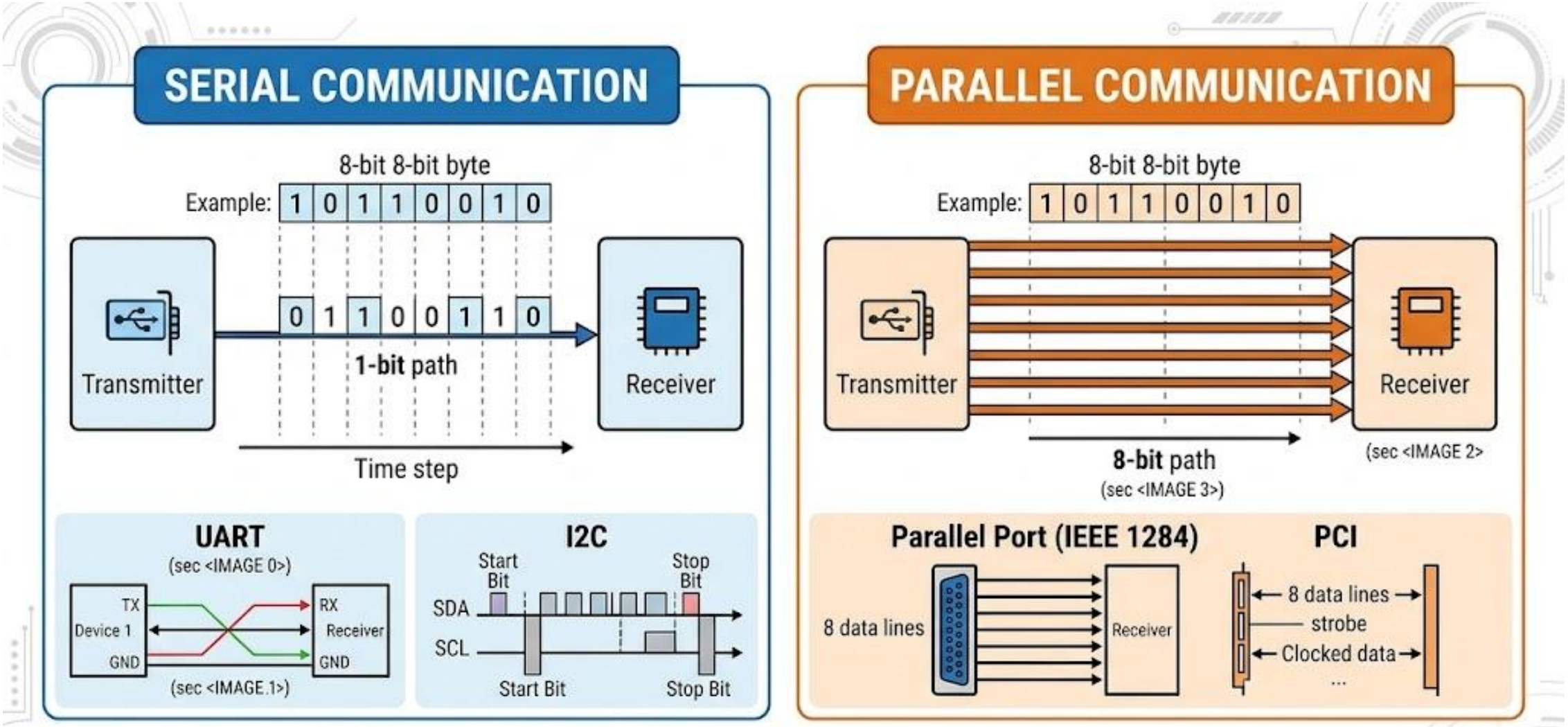
Basics of Communication

- When two electronic devices (like a microcontroller and a computer) need to exchange data, they must agree on **how** and **when** to send bits of information.

- This "conversation method" is called **communication protocol.**



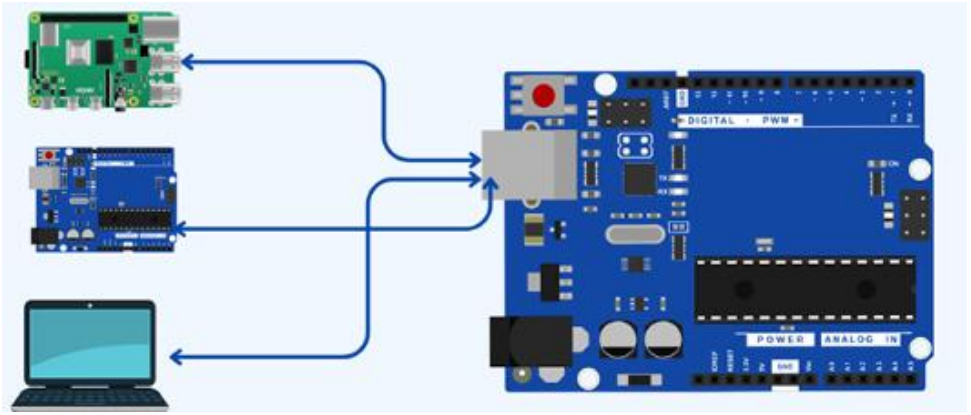
5.1. Introduction to Serial Communication



5.1. Introduction to Serial Communication

Serial Communication

- Data is sent **bit by bit** over a **single wire**.
- **Advantages:**
  - Fewer wires, simpler hardware.
  - Better for long-distance communication.
- **Disadvantages:**
  - Slower than parallel (one bit at a time).
- **Use Case:** Between microcontroller and peripherals, or PC and microcontroller.



5.1. Introduction to Serial Communication

Parallel Communication

- Multiple bits (e.g., 8 bits) are sent **at the same time**, each bit having its **own wire**.
- **Advantages:**
  - High speed (all bits sent together).
- **Disadvantages:**
  - Requires many wires → more cost and space.
  - Difficult to synchronize over long distances (timing differences cause errors).
- **Use Case:** Inside computers, short-distance data transfer (e.g., between CPU and RAM).



5.1. Introduction to Serial Communication

Serial and Parallel Communication Overview

Feature	Serial Transmission	Parallel Transmission
Data Flow	Bits sent sequentially (one after the other) over a single pair of wires (TX/RX).	Multiple bits sent simultaneously over separate wires (e.g., 8 wires for 8 bits).
Speed	Relatively **Slower** bit-by-bit transfer.	Very **Fast** over short distances.
Hardware	**Few** wires required (often just 2 or 3). Simple and cost-effective.	**Many** wires required. Bulky and expensive.
Distance/Skew	**Excellent** for long distances; less prone to skew.	**Poor** over long distances due to **data skew** (bits arrive at different times).
Examples	**USB, Ethernet, I^2C, SPI, UART/USART.**	**Old printer ports, internal data buses.**

5.1. Introduction to Serial Communication

Examples



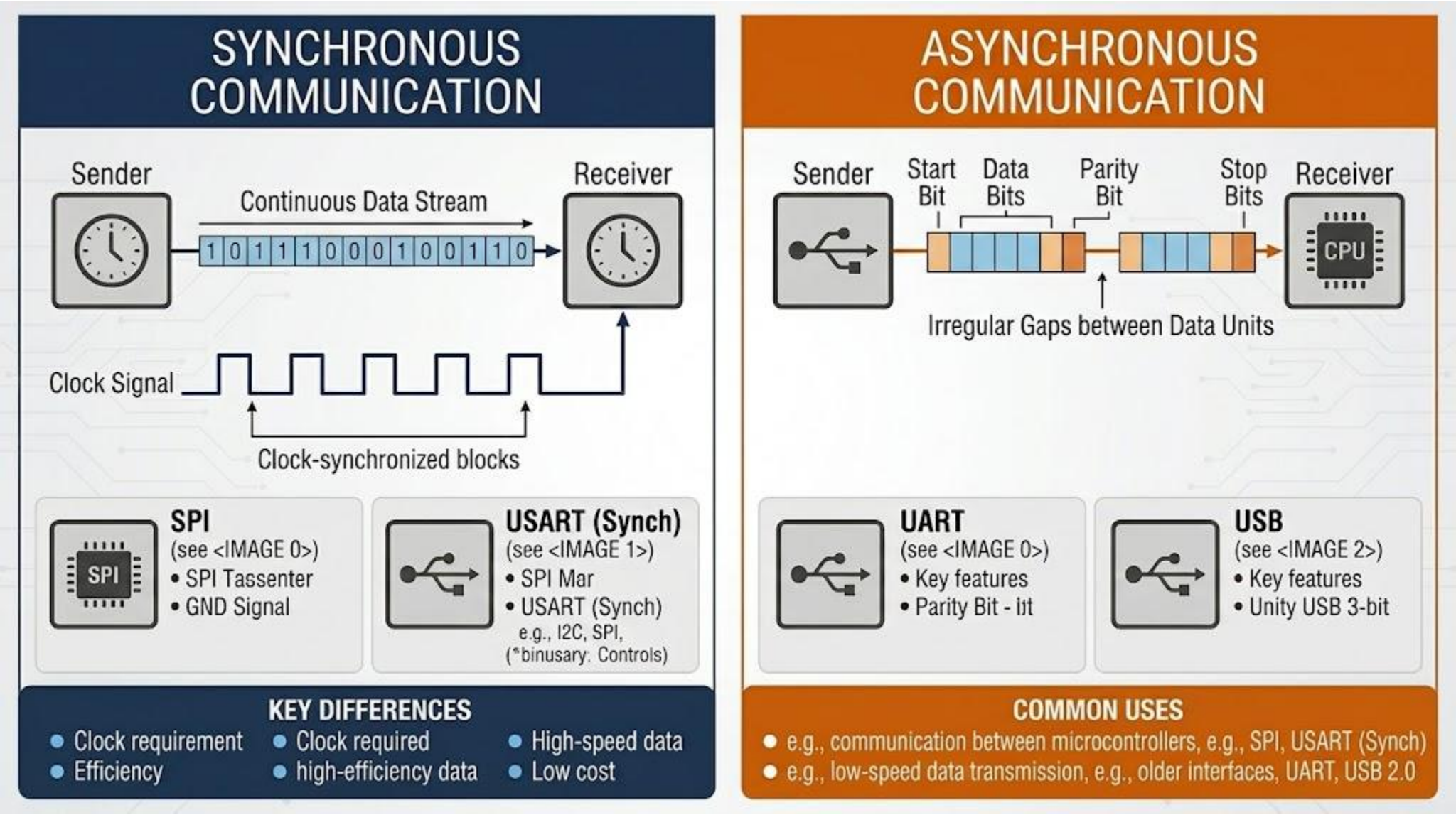








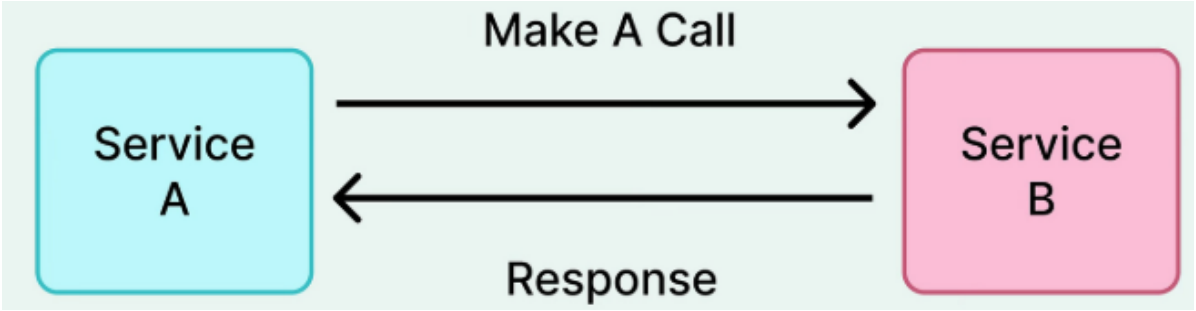
5.1. Introduction to Serial Communication



5.1. Introduction to Serial Communication

Synchronous Communication

- A **shared clock** signal keeps **sender** and **receiver** in sync.
- Both know exactly **when** to send and read each bit.
  - **Example:** SPI (Serial Peripheral Interface)
  - **Advantages:** High data rate and precise timing.
  - **Disadvantages:** Needs an extra wire for the clock signal.
- **Use Case:** Communication between microcontroller and sensors, displays, etc.

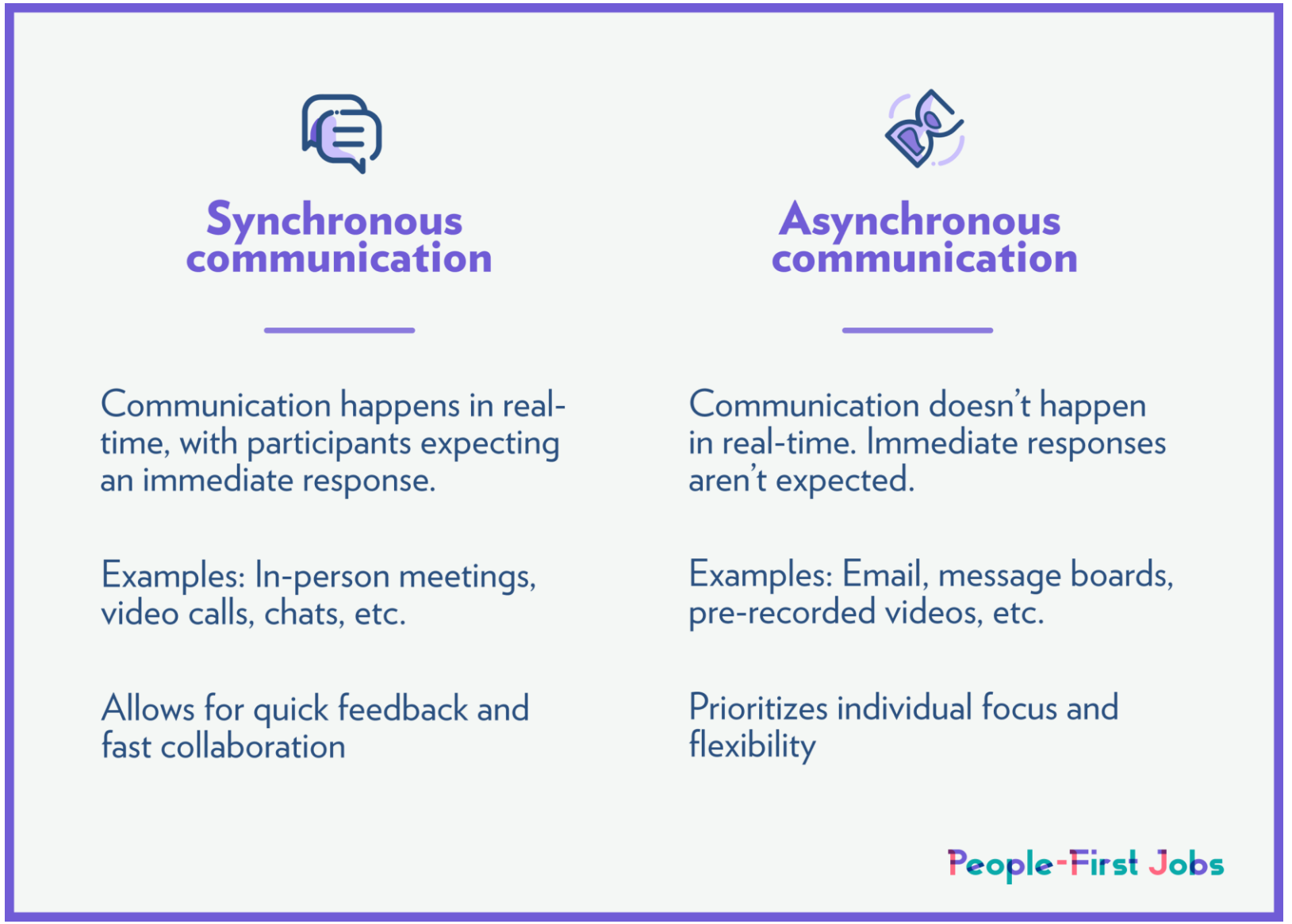


5.1. Introduction to Serial Communication

Asynchronous Communication

- **No shared clock,** each device runs on its **own clock**.
- Special **bits** (Start and Stop bits) are used to mark when a byte begins and ends.
  - **Example:** UART (Universal Asynchronous Receiver Transmitter)
  - **Advantages:**
    - Fewer wires (no clock line).
    - Simple and reliable for slower data transfer.
  - **Disadvantages:** Slight timing mismatch can cause errors.
- **Use Case:** PC to microcontroller serial port, Bluetooth modules.

5.1. Introduction to Serial Communication



5.1. Introduction to Serial Communication

Asynchronous Serial Protocol

- In **asynchronous communication**, data is sent **one character (byte)** at a time **without** a shared clock.
- To make sure both the **sender** and **receiver** stay in sync, each byte is framed with **special control bits**.
- This group of bits forms a **data frame**.

```
| Start Bit | Data Bits | Parity Bit | Stop Bit(s) |
```

5.1. Introduction to Serial Communication

1. Start Bit

- When no data is being sent, the line stays at a **high (logic 1)** level (called **Idle State**).
- To begin transmission, the sender pulls the line **low (logic 0)** for **one bit time**.
- This signals to the receiver: "A new byte is coming, start reading bits now!"

Analogy: Think of it like knocking on someone's door before you start talking.

5.1. Introduction to Serial Communication

2. Data Bits

- These are the **actual data** being sent (the character or number).
- Usually **5 to 9 bits** per frame, most common are **8 bits**.
- Data bits are sent **Least Significant Bit (LSB) first**.

Example: To send the letter 'A' (binary 01000001), bits are sent as:

1, 0, 0, 0, 0, 0, 1, 0 (LSB first)

5.1. Introduction to Serial Communication

3. Parity Bit (Optional)

- The **parity bit** is used for **error checking**.
- It ensures that the total number of 1's in the frame is **even** (even parity) or **odd** (odd parity).
  - **Example:** Data = 01000001 (2 ones)
  - If using **even parity** → parity bit = 0 (total 2 ones = even)
  - If using **odd parity** → parity bit = 1 (makes total 3 ones = odd)
- If sender and receiver counts don't match, there's likely a transmission **error**.

5.1. Introduction to Serial Communication

4. Stop Bit(s)

- After the data (and optional parity), the line is held **high (logic 1)** for **1 or 2 bit times**.
- This tells the receiver that the byte has **finished** and allows it to prepare for the next one.

Analogy: Think of the stop bit like saying "I'm done speaking" before pausing.

5.1. Introduction to Serial Communication

Baud Rate

- The **baud rate** defines **how fast** data is transmitted i.e., **bits per second (bps)**.
- Common baud rates: **9600**, **19200**, **115200 bps**, etc.
- **Example**: At 9600 bps → one bit takes **1 / 9600 ≈ 104 µs**.

Analogy: Both sender and receiver **must use the same baud rate**, or data will be misread (like two people talking at different speeds).

5.1. Introduction to Serial Communication

Example: 8N1 Frame Format

- A very common format in embedded systems is **"8N1"**:
- **8** data bits, **N** = No parity, **1** stop bit
- Total = 1 (start) + 8 (data) + 1 (stop) = **10 bits per character**
- So at **9600 baud**, you can send about:

  9600 / 10 = **960 characters per second**

Contents

5.1. Introduction to Serial Communication
- Basics of Serial Communication
- Asynchronous Serial Protocol

5.2. USART in STM32
- STM32 USART Overview
- USART Block Diagram
- USART Registers

5.3. Serial Communication Programming
- Initialization Steps for USART
- Sending Data (TX) Flow
- Receiving Data (RX) Flow

5.4. Summary

5.2. USART in STM32

What is USART?

- **USART** stands for **Universal Synchronous/Asynchronous Receiver Transmitter**.
- It's a **hardware module** in the STM32 microcontroller that allows it to **send and receive serial data** with other devices. such as computers, sensors, GPS modules, and Bluetooth chips.
  - **UART** → works in **asynchronous mode** (no clock line).
  - **USART** → can work in **both synchronous and asynchronous modes**.

5.2. USART in STM32

Why We Need USART

- In embedded systems, we often need to:
  - **Send** sensor readings to a PC.
  - **Receive** control commands from a terminal.
  - Communicate with modules like GSM, GPS, or Wi-Fi.
- USART makes all of these tasks **simple and reliable** using only **two main lines**:
  - **TX (Transmit)** → sends data out
  - **RX (Receive)** → receives data in



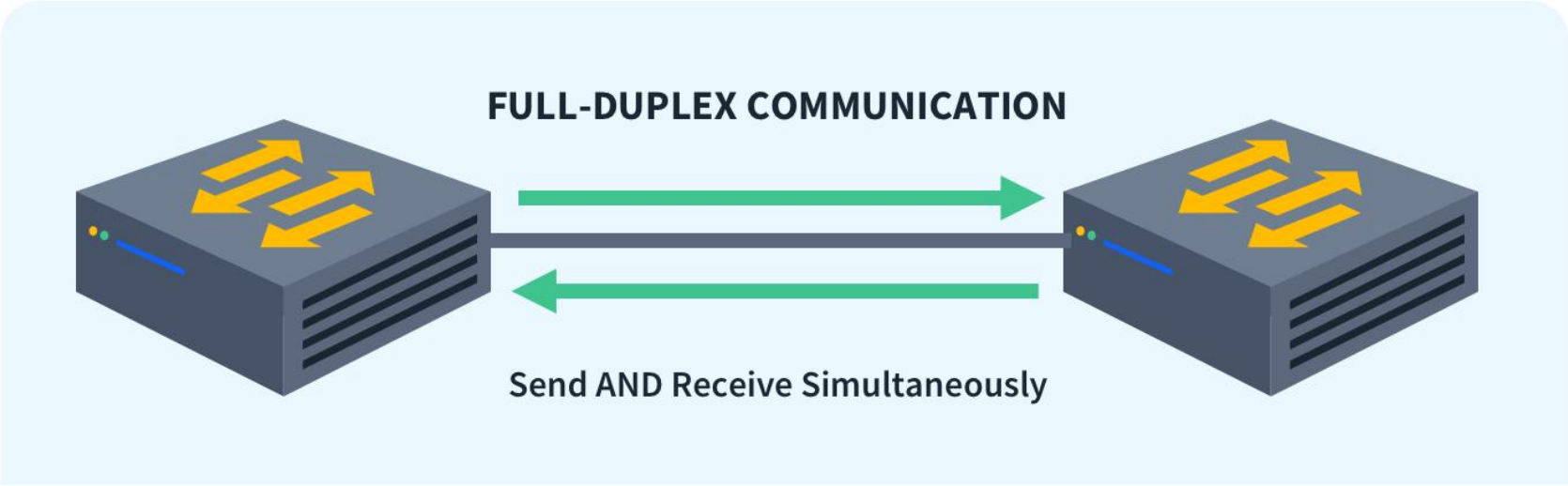
5.2. USART in STM32

Features of STM32 USART

1. Full-Duplex Communication

- Can **send and receive simultaneously** using separate TX and RX lines.
- This makes communication faster and smoother.

Example: The MCU can send debug data while still listening for commands.



5.2. USART in STM32

2. Configurable Baud Rate

- You can set how fast data is transmitted (bits per second).
- Baud rate is set using the **USART_BRR (Baud Rate Register)**.
- Typical values: **9600**, **19200**, **115200 bps**.
- The baud rate depends on the system clock and BRR value.



5.2. USART in STM32

3. Interrupt Support



- USART can generate **interrupts** when:
  - Data is received (RXNE – Receive Not Empty)
  - Data transmission is complete (TC – Transmission Complete)
  - Errors occur (parity, framing, overrun)
- This lets the CPU **handle other tasks** while waiting for serial events.

Example: Instead of polling, if data arrived, the MCU "gets notified" automatically.

5.2. USART in STM32

4. DMA (Direct Memory Access) Support

- DMA allows data to move **directly between memory and USART** without using the CPU.
- Great for **high-speed or continuous data transfer**.

Example: Streaming sensor data to a PC in real time with minimal CPU load.



5.2. USART in STM32

5. Multiple Operation Modes

- **Asynchronous mode** – classic UART communication (no clock).
- **Synchronous mode** – adds a clock line for faster, synchronized transfer.
- **Single-wire half-duplex mode** – TX and RX share one wire.
- **Smartcard mode / IrDA mode** – for special protocols.

5.2. USART in STM32

6. Error Detection and Parity Options

- Built-in **parity check**, **framing**, and **overrun error detection** help ensure data integrity.
- Useful when communicating over longer or noisy cables.



5.2. USART in STM32

Common Use Cases for STM32 USART

Use Case	Description	Setup
Debug Console	Connecting the STM32 to a PC via a USB-to-Serial adapter (or ST-Link Virtual COM Port) to print **status messages** and receive **commands**.	Asynchronous, 115200 Baud, 8N1 (8 data bits, No parity, 1 stop bit).
External Device Interface	Communicating with a variety of external modules, such as:	Asynchronous, rate and frame depend on the module's specification.
	- **GPS Receivers** (sending configuration, receiving location data)	
	- **GSM/GPRS Modules** (sending AT commands for cellular communication)	
	- **Bluetooth/WiFi Modules** (configuration and data exchange)	
Inter-MCU Communication	Connecting two different microcontrollers directly for simple data exchange.	Asynchronous or **Synchronous** (if using the clock wire) for higher speed.
PC Firmware Update	Using a bootloader to receive new firmware code from a PC for flashing onto the STM32 (involving DMA).	Asynchronous, often at the highest reliable Baud Rate.

5.2. USART in STM32

Example in Real Life

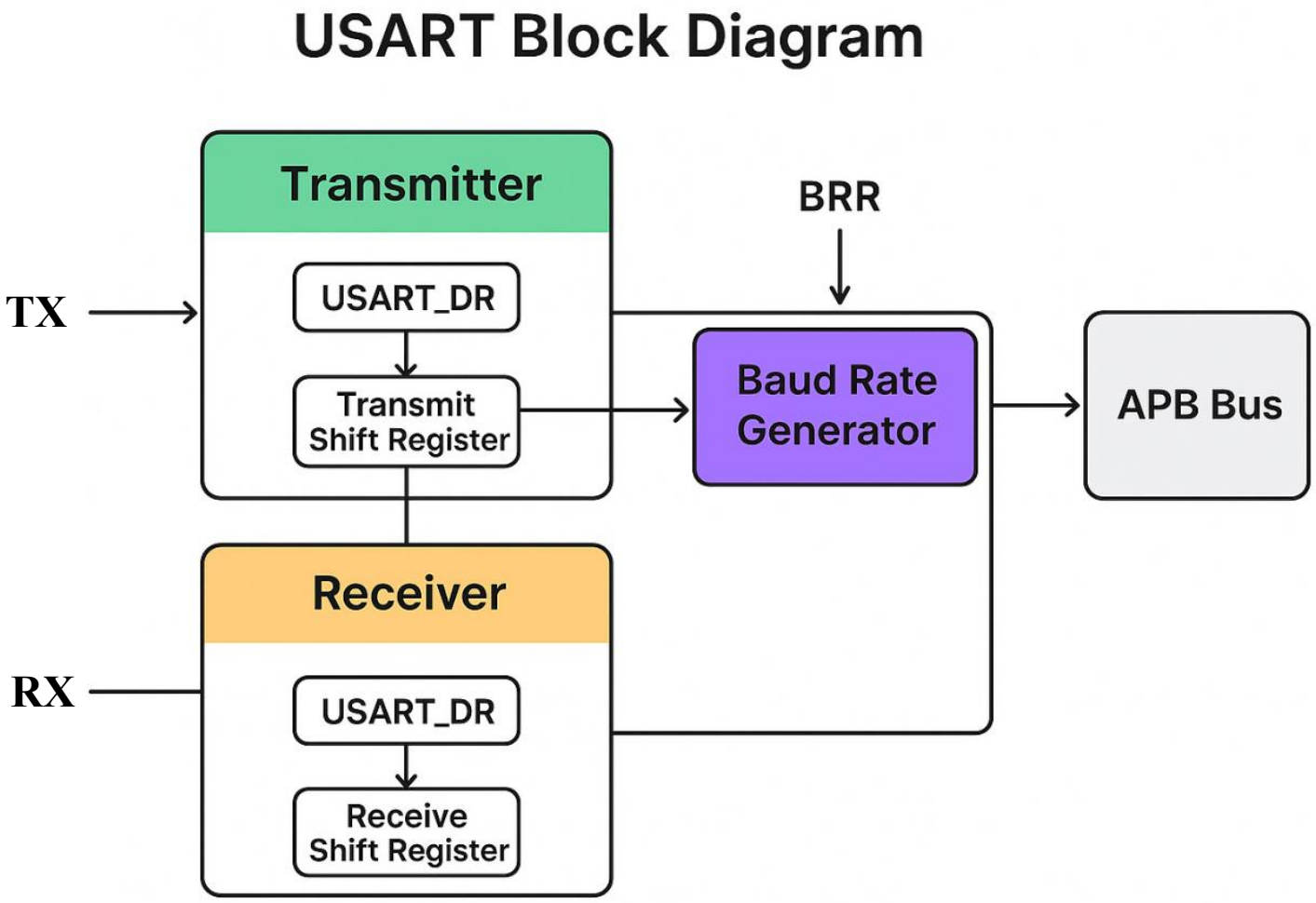
- STM32 is talking to your laptop via USB-to-Serial cable:
- STM32 TX → sends "Temperature: 25°C"
- STM32 RX ← receives "Turn on Fan"
- **USART communication**— sending and receiving text messages between devices.

5.2. USART in STM32

USART Block Diagram (STM32)

- The **USART peripheral** in STM32 is a mini communication engine inside the microcontroller.
- It connects to the CPU through the **APB (Advanced Peripheral Bus)** and handles all the work needed to send and receive data serially — bit by bit — through the **TX** and **RX** pins.

5.2. USART in STM32



5.2. USART in STM32

1. Transmitter (TX Section)

- Responsible for **sending data** from the STM32 to an external device.
- Works like a delivery system, it takes a byte from memory or register and ships it out bit by bit.

Steps:

1. The CPU writes data into the **Data Register (USART_DR)**.
2. The **Transmit Shift Register (TSR)** takes that byte.
3. The bits are sent serially through the **TX pin,** starting with the **Start Bit**, followed by **Data Bits**, optional **Parity Bit**, and **Stop Bit(s)**.

- Once sending is complete, the USART sets a **Transmission Complete (TC)** flag or triggers an **interrupt/DMA request**.

5.2. USART in STM32

2. Receiver (RX Section)

- Responsible for **receiving serial data** from an external device (like a PC).
- Works like a mailbox, it collects bits one by one and assembles them into a full byte.
- **Steps:**
  1. The **RX pin** detects the **Start Bit** (line goes low).
  2. Bits are shifted into the **Receive Shift Register (RSR)** at the correct timing.
  3. Once a full byte is received, it moves into the **Data Register (USART_DR)**.
  4. The USART sets the **RXNE (Receive Not Empty)** flag to tell the CPU new data is ready.
- The CPU can then read the byte directly or handle it using an interrupt/DMA.

5.2. USART in STM32

3. Baud Rate Generator

- Controls **how fast** bits are sent and received.
- It uses the **main system clock (e.g., 72 MHz)** divided by a programmable value in the **USART_BRR register** to produce the desired baud rate.
- **Example:**

  If system clock = 72 MHz and baud rate = 9600 bps

  Then BRR = 72,000,000 / (16 × 9600) ≈ 468.75
- The USART hardware uses this to generate precise timing for bit transmission and reception.
- This ensures both sender and receiver stay "in rhythm" even without a shared clock line (in asynchronous mode).

5.2. USART in STM32

4. Shift Registers

- There are **two shift registers**:
- **Transmit Shift Register (TSR)** – converts **parallel** data (byte from memory) into **serial** stream for TX.
- **Receive Shift Register (RSR)** – converts **serial** bits from RX line into **parallel** data (byte).
- These registers handle the "bit-by-bit" shifting process — that's why we call it **serial communication**!

5.2. USART in STM32

5. Connection to APB Bus

- The USART module is connected to the **APB (Advanced Peripheral Bus)** inside STM32.
- The **CPU accesses USART registers** (like CR1, CR2, BRR, DR, SR) through this bus.
- All control commands, data writes, and status reads happen via the **APB clock domain**.
- In STM32, peripherals like USART, GPIO, and TIMERS are all mapped on the APB1 or APB2 buses.
- **Example**:
  - **USART1** → on **APB2 bus**
  - **USART2/3** → on **APB1 bus**

5.2. USART in STM32

USART Registers in STM32

- To control the USART peripheral, the STM32 microcontroller uses several **memory-mapped registers**.
- Each register stores specific configuration data, status information, or the actual data being sent or received.
- Understanding these registers is key to programming the USART manually (without using library functions).

5.2. USART in STM32

Overview of USART Registers in STM32

Register	Purpose	Direction
USART_SR	Status register — shows what's happening	Read
USART_DR	Data register — holds transmit/receive data	Read/Write
USART_BRR	Baud rate register — sets communication speed	Write
USART_CR1 / CR2 / CR3	Control registers — configure USART behavior	Write

5.2. USART in STM32

1. USART_SR – Status Register

- This register shows the **current state of the USART** — whether data is ready, transmission complete, or if any error occurred.

Common Bits: Before writing data, check if TXE = 1; before reading, check if RXNE = 1.

Bit	Name	Meaning
TXE (Bit 7)	Transmit Data Register Empty	1 → Ready to send next byte
TC (Bit 6)	Transmission Complete	1 → Last byte fully sent
RXNE (Bit 5)	Read Data Register Not Empty	1 → New data received
ORE (Bit 3)	Overrun Error	Data lost (receiver too slow)
FE (Bit 1)	Framing Error	Wrong start/stop bit detected
PE (Bit 0)	Parity Error	Parity check failed

5.2. USART in STM32

2. USART_DR – Data Register

- The **data buffer** between the CPU and USART hardware.
- Used for both **transmitting** and **receiving** data.
- **Usage:**
  - Write to USART_DR → to **send** a byte.
  - Read from USART_DR → to **receive** a byte.

```
USART1->DR = 'A';      // Send character 'A'
char data = USART1->DR; // Read received data
```

5.2. USART in STM32

3. USART_BRR – Baud Rate Register

- This register defines the **baud rate (communication speed)**.
- It divides the peripheral clock to generate bit timing for serial communication.
- Higher baud rate → faster communication but more error-sensitive.

5.2. USART in STM32

4. USART_CR1 – Control Register 1

- Controls the **basic functions** of the USART.

Bit	Name	Function
UE (Bit 13)	USART Enable	1 → Turn on USART
M (Bit 12)	Word Length	0 = 8-bit, 1 = 9-bit
PCE (Bit 10)	Parity Control Enable	1 → Enable parity checking
PS (Bit 9)	Parity Selection	0 = Even, 1 = Odd
TE (Bit 3)	Transmitter Enable	1 → Enable TX
RE (Bit 2)	Receiver Enable	1 → Enable RX
RXNEIE (Bit 5)	RX Interrupt Enable	1 → Trigger interrupt on received data

5.2. USART in STM32

5. USART_CR2 – Control Register 2

- Used for **frame format** and **synchronization features**.

Bit	Name	Function
STOP (13:12)	Stop Bits	00 = 1 bit, 10 = 2 bits
CLKEN (Bit 11)	Clock Enable (Synchronous mode)	1 → Enable clock output
CPOL / CPHA	Clock Polarity / Phase	Used in synchronous mode
LINEN	LIN Mode Enable	For LIN protocol support

5.2. USART in STM32

6. USART_CR3 – Control Register 3

- Adds **advanced features** like DMA and hardware flow control.
- DMA and flow control are used in high-speed or continuous data transfer applications.

Bit	Name	Function
DMAT (Bit 7)	DMA Enable for Transmit	1 → TX via DMA
DMAR (Bit 6)	DMA Enable for Receive	1 → RX via DMA
RTSE / CTSE	Hardware Flow Control	Enable RTS/CTS
EIE	Error Interrupt Enable	Generate interrupt on errors

Contents

5.1. Introduction to Serial Communication
- Basics of Serial Communication
- Asynchronous Serial Protocol

5.2. USART in STM32
- STM32 USART Overview
- USART Block Diagram
- USART Registers

5.3. Serial Communication Programming
- Initialization Steps for USART
- Sending Data (TX) Flow
- Receiving Data (RX) Flow

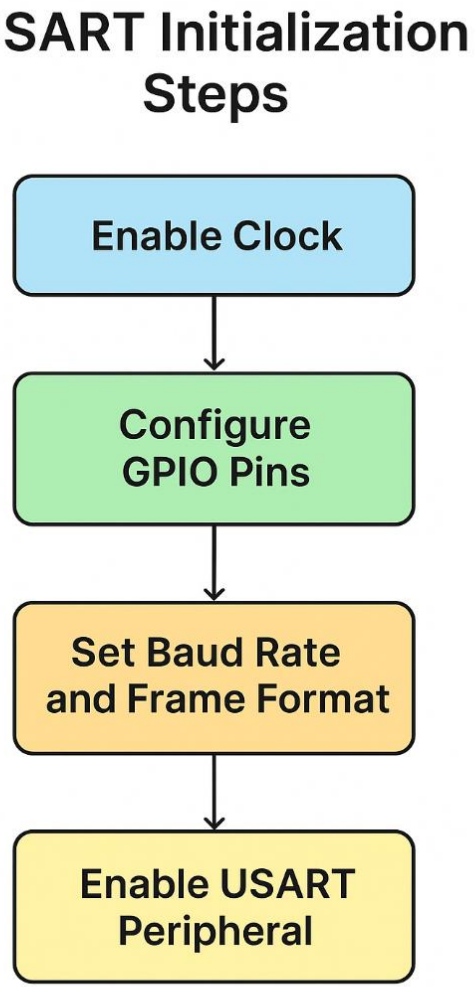
5.4. Summary

5.3. Serial Communication Programming

Serial Communication Programming (USART Initialization)

Before sending or receiving data through the serial port, we must **initialize** the USART peripheral:

- **which pins to use**,
- **how fast to communicate**,
- **what format to send data in**.



5.3. Serial Communication Programming

Step 1: Enable Clock (RCC)

- All peripherals in STM32 (GPIO, USART, TIMERS, etc.) need their **clock enabled** before use.

  This is done through the **RCC (Reset and Clock Control)** module.

- **Why?**
  - Without enabling the clock, the peripheral's registers are inactive.
  - Think of it like powering up a device before configuring it.
- **Example:** If using **USART1** and **GPIOA** (for TX and RX pins):

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;    // Enable GPIOA clock
RCC->APB2ENR |= RCC_APB2ENR_USART1EN; // Enable USART1 clock
```

5.3. Serial Communication Programming

Step 2: Configure GPIO Pins (TX and RX)

- USART uses two main pins:
- **TX (Transmit)** – sends data
- **RX (Receive)** – receives data
- Configure these pins to act as **Alternate Function (AF)** pins instead of general-purpose I/O.
- **Example (USART1 on GPIOA):**

```
GPIOA->CRH &= ~(0xFF << 4);      // Clear PA9, PA10
GPIOA->CRH |= (0x0B << 4);       // PA9 = AF Push-Pull, 50 MHz
GPIOA->CRH |= (0x04 << 8);       // PA10 = Input floating
```

Without this, data won't appear on the physical pins even if USART is enabled.

5.3. Serial Communication Programming

Step 3: Configure Baud Rate and Frame Format

- **Frame Format (USART_CR1, CR2)**
- **Word length**: 8 bits or 9 bits
- **Stop bits**: 1 or 2
- **Parity**: None, even, or odd

Example for 72 MHz clock, 9600 baud:

```
USART1->BRR = 0x1D4C;    // 9600 baud @ 72MHz
```

Typical 8N1 configuration:

"8N1" means: **8 data bits, No parity, 1 stop bit** — the most common setup for UART.

```
USART1->CR1 &= ~USART_CR1_M;      // 8-bit data
USART1->CR2 &= ~USART_CR2_STOP;   // 1 stop bit
USART1->CR1 &= ~USART_CR1_PCE;    // No parity
```

5.3. Serial Communication Programming

Step 4: Enable USART Peripheral

- Finally, turn on the USART module and enable transmission/reception.

```
USART1->CR1 |= USART_CR1_TE;     // Enable Transmitter
USART1->CR1 |= USART_CR1_RE;     // Enable Receiver
USART1->CR1 |= USART_CR1_UE;     // Enable USART (main switch)
```

- Without setting UE, the USART hardware remains disabled even if everything else is configured.

5.3. Serial Communication Programming

Implementing USART on the STM32

- **Hardware Setup:** Identify the TX and RX pins for the chosen USART peripheral on the STM32 datasheet.
- **Initialization (Software):**
  - **Enable Clock:** Power the USART peripheral clock (e.g., using RCC registers).
  - **Configure GPIO:** Set up the TX and RX pins for **Alternate Function** mode.
  - **Set Baud Rate:** Write the calculated value to the USART_BRR register.
  - **Configure Frame:** Set data length (e.g., 8 bits), parity (None/Even/Odd), and stop bits (1 or 2).
  - **Enable USART:** Set the UE bit in the Control Register.

Contents

5.1. Introduction to Serial Communication
- Basics of Serial Communication
- Asynchronous Serial Protocol

5.2. USART in STM32
- STM32 USART Overview
- USART Block Diagram
- USART Registers

5.3. Serial Communication Programming
- Initialization Steps for USART
- Sending Data (TX) Flow
- Receiving Data (RX) Flow

5.4. Summary

5.4. Summary

Basics of Serial Communication

- Data is sent **bit by bit** through one wire (serial), instead of many wires (parallel).
- **Synchronous**: uses a shared clock (e.g., SPI).
- **Asynchronous**: no shared clock, uses start/stop bits (e.g., UART/USART).

Asynchronous Serial Protocol

- **Frame format:** Start bit → Data bits → (Optional) Parity → Stop bit(s).
- **Baud rate:** number of bits transmitted per second (e.g., 9600 bps).
- Both sender and receiver must use the **same baud rate**.

5.4. Summary

STM32 USART Overview

- USART = **Universal Synchronous/Asynchronous Receiver Transmitter**.
- Supports **asynchronous**, **synchronous**, and **half-duplex** modes.
- **Full-duplex**: can send and receive simultaneously.
- Features: configurable baud rate, interrupts, DMA support, error detection.
- Common uses: debugging (printf), module communication (GPS, Bluetooth, etc.).

5.4. Summary

USART Block Diagram (Main Components)

- **Transmitter (TX):** sends data bit by bit.
- **Receiver (RX):** receives serial data and stores it in data register.
- **Baud Rate Generator:** sets communication speed.
- **Shift Registers:** convert parallel ↔ serial data.
- **APB Bus Connection:** links USART registers to CPU

5.4. Summary

Important USART Registers

- **USART_SR:** Status (TXE, RXNE, TC, errors).
- **USART_DR:** Data (read/write buffer).
- **USART_BRR:** Baud rate configuration.
- **USART_CR1/CR2/CR3:** Control registers (enable, parity, stop bits, DMA, etc.).

